

电子电路实验

智能元件参数测试仪报告

王凯灵 521030910356

张杭磊 521030910358

代铭波 519030910364

2022 年 12 月 12 日

1 原选题工作

本组原来选择了超声波的题目。此前已经完成了超声波发生电路。电路采用 MAX232 芯片，基本电路如图 1 所示。

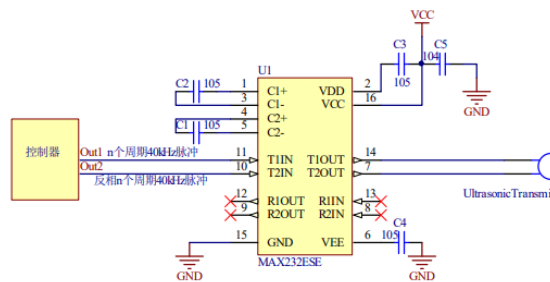


图 1: 超声波发生电路图

其中，五个电容均使用了 $0.1\mu F$ 。输入信号使用了 40kHz 的方波信号，vpp 设置为 3.5V (即芯片最低标准输入电压)。芯片和电路在面包板上的连线实物图如图 4 所示，近距离共地示波器波形如 6 所示，可以看出示波器钩子接收到了比较平整的同频率超声波方波信号。

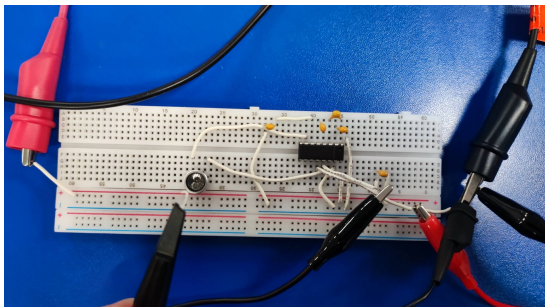


图 2: 超声波发生电路实物图



图 3: 超声波示波器图像

调整示波器接收端与发生器的距离。接收到的信号强弱随距离明显变化，首先于人类右臂长度，最远距离达到约一米，仍可以观察到信号。

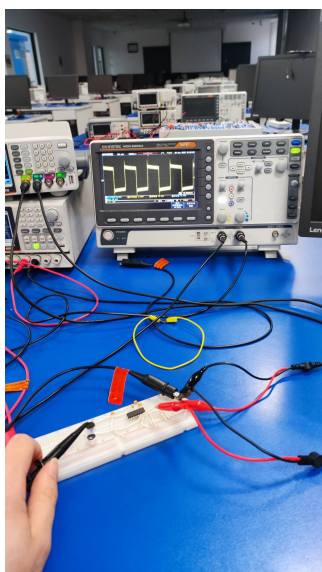


图 4: 近距离

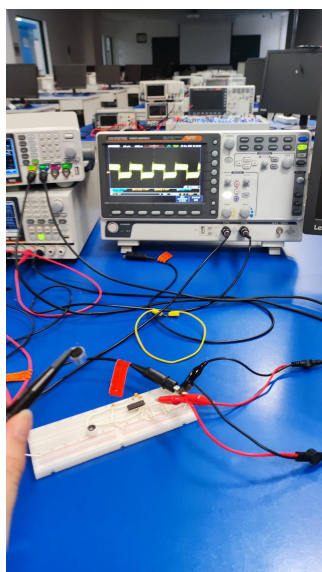


图 5: 中距离

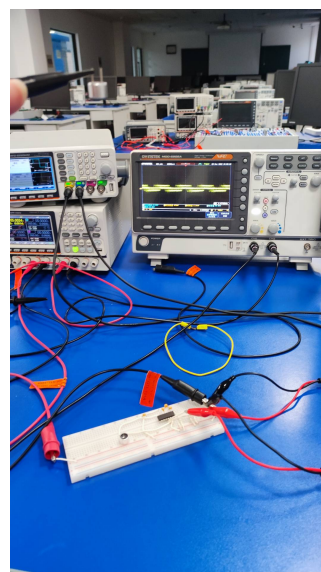


图 6: 远距离

2 摘要

我们实现了智能万用表的基本测量要求，在此基础上使用 ESP8266 模块实现了与服务器的 TCP 协议 WIFI 连接，并实现了时间显示功能和 log 记录功能。

2.1 使用器件表

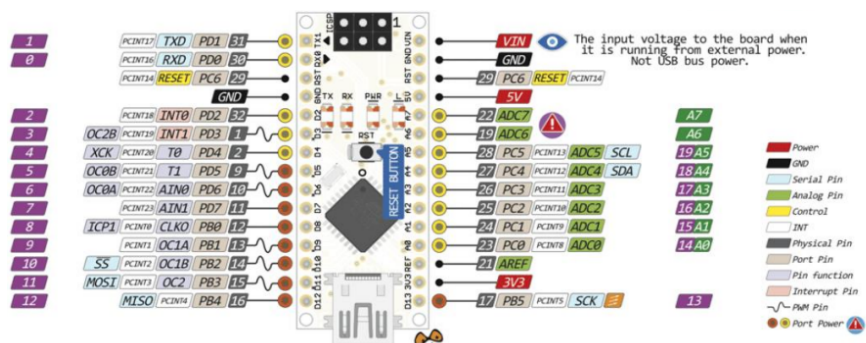


图 7: Arduino NANO 端口功能

使用的器件有：Arduino Nano 单片机若干，拓展板一块，Nokia5100 LCD 显示器一块，杜邦线若干，开关一个，面包板两块，待测元件若干，ESP8266 模块若干，6.7V 可充电锂电池一块，服务器主机和路由器设备。

3 原理说明

3.1 电阻测量

3.1.1 基本原理

已知阻值 R_0 , 测量未知电阻 R_x , 可利用电阻分压规律来得到 R_x :

$$\frac{U_{R_x}}{V_{CC}} = \frac{R_x}{R_x + R_0}$$
$$R_x = \frac{U_{R_x} R_0}{V_{CC} - U_{R_x}}$$

下面讨论如何选取适合的电阻值来配合未知电阻。

3.1.2 设计思路

利用已知两类阻值的电阻, 我们要选取合适的阻值测量未知的 R_x 。对于阻值大的未知电阻, 我们应当选择大值 R_0 , 否则理论公式中 U_{R_x} 会接近于 0, 测量精确度低; 小的未知电阻则应选小值 R_0 , 否则理论公式中 U_{R_x} 会接近于 V_{CC} , 也使得测量不准。

我们可通过预测量的方式估计未知电阻的大小, 再自动转入合适的档位进行精确的测量。当 $R_x = 35490.84\Omega$ 时两档所测 R_x 相对误差相等, 在 $V_{CC} = 5V$ 时对应 $R_0 = 680\Omega$ 的电压 $U_{R_x} = 4.818V$ 。则首先在输入端选用小的 $R_0 = 680\Omega$, 用一个 ADC 接口测量未知电阻端电压, 若端电压 $U_{R_x} > 4.818V$ 则电阻阻值较大, 换用 $R_0 = 470k\Omega$ 档位进行测量。

3.2 电容测量

3.2.1 基本原理

利用下图的一阶 RC 电路, 我们能通过电路方程对于电容值进行测量

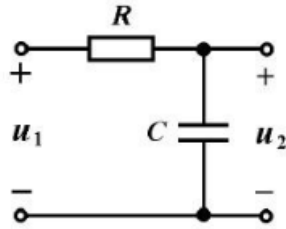


图 8: 基本一阶 RC 电路

给予电压 $U_1 = V_{cc}$, 我们利用下述公式求得电容值:

$$U_2 = U_C = V_{cc}(1 - e^{-\frac{t}{RC}})$$

该公式主要用途是可通过充电时间与充电电压的关系来确定电容 C 的值。如对于小电容, 为电路安全考虑不能充电太久, 则采用“充电电压一定, 测充电时间”的方式测定电容值; 对于大电容, 由于充电效率较低, 可以采用“充电时间一定, 测充电电压”的方式来测定电容值。在下一部分中将讨论方式的确定与参数的选取。

3.2.2 设计思路

对于测定电容端电压，方式类似电阻。同样选用一个 ADC 接口：

对于电阻选取，我们应尽可能让充电时间长，这样不论是充电时间一定时还是充电电压一定时，对数值的读取都会更为精确；当然充电时长也不应过长。则根据时间常数 RC 确定，对于大电容应当选取小电阻，小电容应当选取大电阻。

若认为 $t=5\tau$ 时完成充放电，则只需测充电时间就能计算出电容 C 。由于 Arduino 时钟频率为 16MHz，时钟周期约为 63ns，则 τ 应至少在 μs 数量级才能有较好的测量效果。可先选取小电阻进行测量，若充电时间过短再切换大电阻。

3.3 三极管测量

3.3.1 基本原理

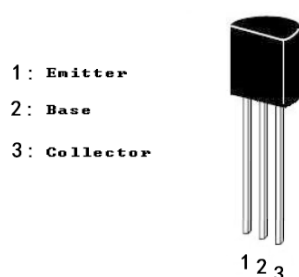


图 9: 三极管示意图

三极管作为一个不对称元件，与之前的元件很大不同在于需要对不同端口的特性进行识别并做出选择。

3.3.2 双极管类型与电极的确定

首先我们应明确三极管的结构，对于一个双极晶体管，我们只可能从 b 极分别往 c 极和 e 极加电压时才有可能导通。并且对于 PNP 型的管， b 极应当是低电位；对于 NPN 型的管， b 极应当是高电位。

假设现有一枚 NPN 型双极管，我们可以对三级分别接入高电位进行 b 极的确定。当我们接高电压的极为 b 时，则两侧均能导通；若为 c 或 e 极，则两侧均截止。对于 PNP 管原理是一样的，但用的是低电位，同样，若接到 b 极则能检测到两侧导通，接到 c 或 e 极则一侧导通，另一侧截止。

通过这样的测试，我们能确定出管子的类型以及其 b 极的位置。接下来我们应确定 c ， e 极。根据三极管的物理特性，我们知道集电极具有较高的掺杂浓度，当给予的电压相同时， bc 具有比 be 更小的电阻与更大的电流。故根据已确定的 PNP(NPN)，分别接低(高)电压，判断两端导通电流大者为 c ，小者为 e 。

3.3.3 参数 β 与 U_{be} 的测量

下面以 NPN 管为例说明参数的测量方式。下图是被采用的测量电路，

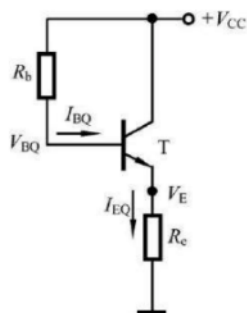


图 10: 基本一阶 RC 电路

其中 $R_b = 510k\Omega$, $R_e = 680\Omega$ (电阻阻值的选择是根据导通后两电阻的分压应当量级接近而进行的) 根据有关模电知识计算可得:

$$U_{BE} = V_{BQ} - V_E$$

$$\beta = \frac{V_E/R_e}{(V_{CC} - V_{BQ}/R_b)}$$

对 PNP 管, 测量方式与 NPN 管类似。

3.3.4 设计思路

对于工作在饱和区的三极管, 有 V_b 与 V_e 的差约为 0.3V 0.7V。对于 NPN 型三极管有 $V_b > V_c > V_e$, PNP 型三极管有 $V_e > V_c > V_b$ 。在实际测量过程中可依次让一个端口接高电位, 另外两个端口接低电位, 观察三极管何时满足上述条件, 再根据电压之间的关系就能测出三极管的端口信息。

4 仿真模拟

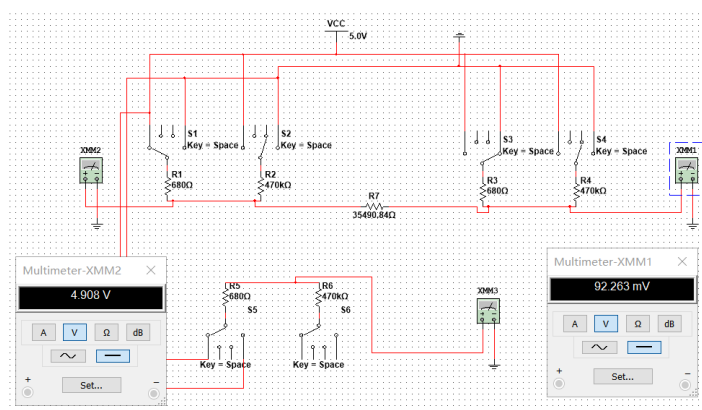


图 11: 电阻测量电路

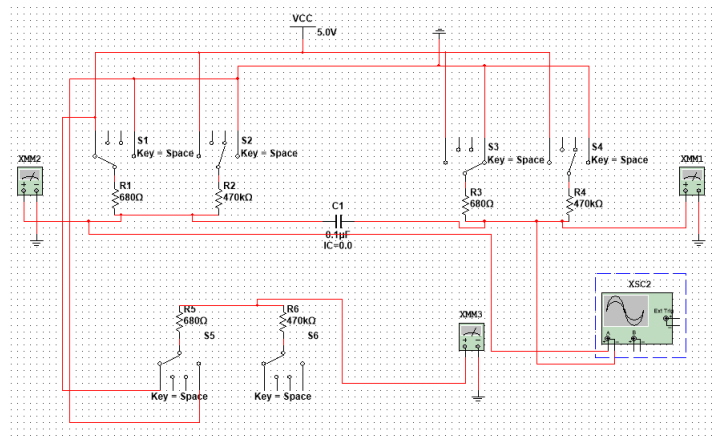


图 12: 电容测量电路

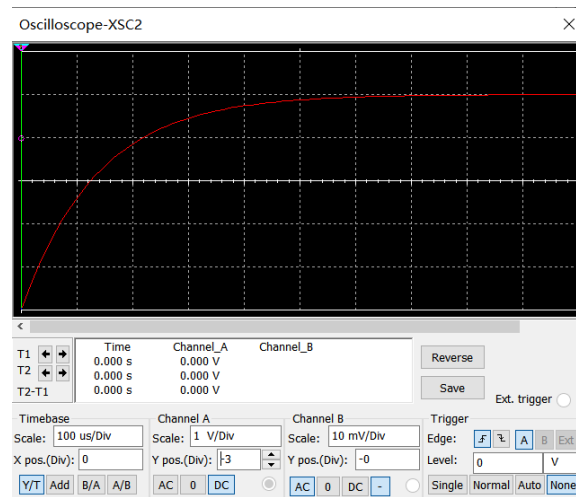


图 13: 电容充电曲线

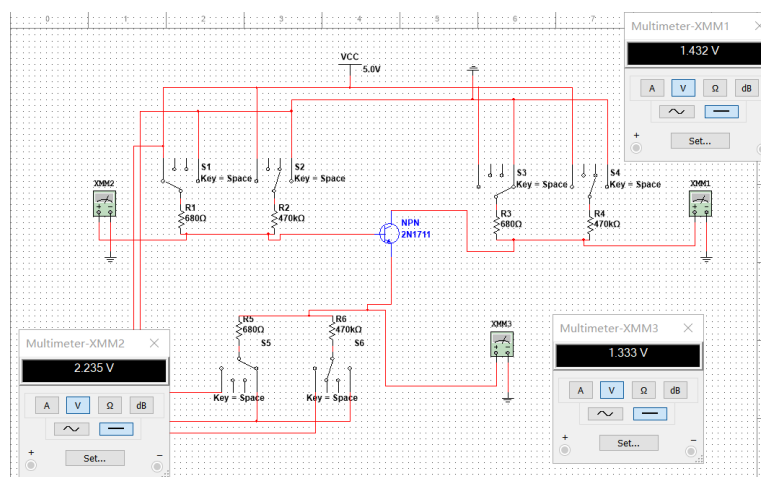


图 14: NPN 型三极管

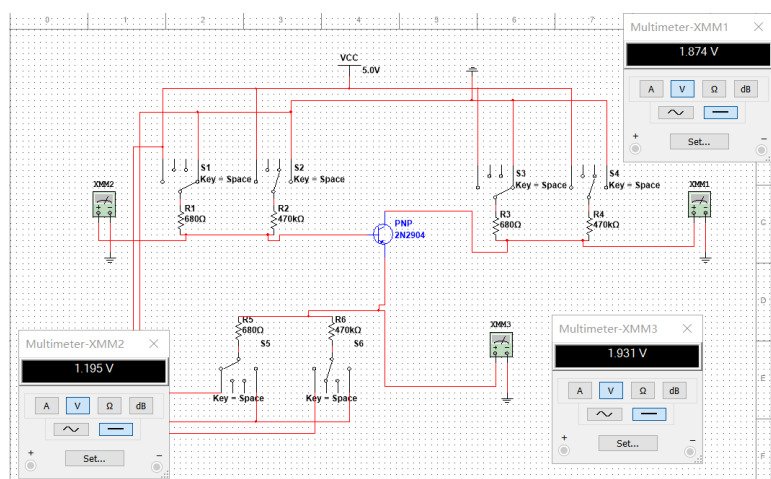


图 15: PNP 型三极管

5 电路和代码实现

5.1 基本功能

本作业中，所需实现的功能主要有：电阻参数的测量，电容参数的测量，三极管参数的测量。

图 16是我们的成果实物图。

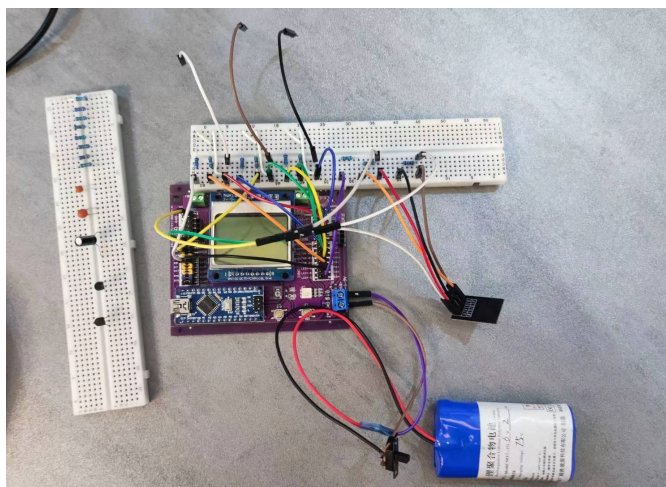


图 16: 成果实物图

主体部分使用了拓展板进行接线。拓展板仅作为接线柱和供电使用，我们没有使用拓展板的任何其他功能，包括屏幕跳线也是重新定义的。屏幕是原选题使用的 LCD 屏幕。

此外，我们还使用了锂电池供电和无线网络模块，会在之后的部分进行说明。

面板版上连接了通过杜邦线引出来的读写端口和电阻。单个测试节点包含一个具有 ADC 采样功能的端口和两个可写端口，其中可写端口分别连接 680Ω 和 $470k\Omega$ 的电阻，最后并联引出测试点。测试点电路如图 17所示。由于三个测试点是对称的，我们将面板版上的连线也做到了对称和美观，电阻位置和测试点引线十分清晰。

我们的端口定义如表 1所示。

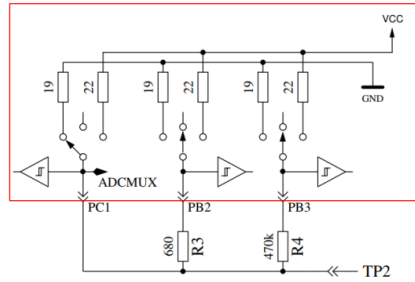


图 17: 测试点电路图

表 1: 端口定义

端口号	功能	用途
D14	ADC0	TP1 读取
D8	PB0	TP1 电平
D9	PB1	TP1 电平
D15	ADC1	TP2 读取
D12	PB4	TP2 电平
D13	PB5	TP2 电平
D16	ADC2	TP3 读取
D10	PB2	TP3 电平
D11	PB3	TP3 电平
D6	PD6	显示控制
D3	PD3	8266TX
D2	PD2	8266RX

5.2 电阻测量实现

如何确定测量使用的 TP 点将在控制部分阐述。此处均假设已知测试点信息和所需测量元件类型。

实现电阻的测量，首先假设电阻较小。即：将 $470k\Omega$ 连接的端口悬空，将 680Ω 连接的端口电平写为一高一低，使用两个 ADC 端口的采样结果计算外接电阻上的电压差。ADC 读取结果为一个 10 比特位表示的无符号整数，即取值在 $[0,1023]$ 。最高电压为 5V，通过简单的线性关系就可以得到电压的采样值。

使用如下函数完成读取电压差值的操作。

```

1 // test the case for small resistance
2 double testRS(int x, int y, int cmd=1)
3 {
4     setLow();
5     pinMode(TP[x][ADC], INPUT);
6     pinMode(TP[y][ADC], INPUT);
7     pinMode(TP[x][R1], INPUT);
8     pinMode(TP[y][R1], INPUT);
9     pinMode(TP[x][Rs], OUTPUT);
10    pinMode(TP[y][Rs], OUTPUT);

```

```

11 digitalWrite(TP[x][Rs], HIGH);
12 digitalWrite(TP[y][Rs], LOW);
13 delay(5);
14 int vintH=analogRead(TP[x][ADC]);
15 int vintL=analogRead(TP[y][ADC]);
16 if(abs(vintH+vintL-1023)>5 and cmd)
17     return 114;
18 double vdiff = (vintH-vintL)*5.0/1023;
19 return vdiff;
20 }

```

通过分压关系可以得到对应的电阻值。

测量较大电阻时，换用大电阻进行分压即可。

5.3 电容测量实现

测量电容时，需要首先稳定两端电压为 0。从此时开始计时，并将一端电压设为高电平。测量大电容是，选用小电阻，测量小电容时，使用大电阻。一段时间后，ADC 采样获取电压差值达到某一设定好的电压，此时停止计时。时间长度为约 $10RC$ 左右，以此计算电容值。具体充电时间与设定好的到达电压直接相关，与温度等其他因素也有关联。

以下的代码可以用于测量较大电容。

```

1 // large capacity
2 String testCL(int x, int y)
3 {
4     setLow();
5     disp(String("Mode: LC"), 0, 1); // Small C
6     disp(String("Testing"), 0, 2);
7     disp(String("Don't shift!"), 0, 3); // Warning
8     pinMode(TP[x][ADC], INPUT);
9     pinMode(TP[y][ADC], INPUT);
10    pinMode(TP[x][R1], INPUT); // use small R, the charge time is RC. If C is
        too small, charge time is small. This case goto TestCS to use larger R
11    pinMode(TP[y][R1], INPUT);
12    pinMode(TP[x][Rs], OUTPUT);
13    pinMode(TP[y][Rs], OUTPUT);
14    int rcdv[200];
15    String tmpS;
16    int cnt=0;
17    int vintH=analogRead(TP[x][ADC]);
18    int vintL=analogRead(TP[y][ADC]);
19    while(vintH>5 or vintL>5)
20        vintH=analogRead(TP[x][ADC]), vintL=analogRead(TP[y][ADC]); // wait
        until q=0
21    delay(10);
22    long int Cstart=micros();
23    digitalWrite(TP[x][Rs], HIGH);
24    while(analogRead(TP[x][ADC])<1018 and micros()-Cstart<1000000)

```

```

25     rcdv[max(cnt++/3,200)]=analogRead(TP[x][ADC]);
26     long int tdiff=micros()-Cstart-116; // 116 is the base
27     tdiff=abs(tdiff);
28     disp("          ", 0, 3);
29     disp("Used "+String(tdiff)+"us",0,3);
30     if(tdiff>900000)
31     {
32         disp("          ", 0, 0);
33         disp("          ", 0, 1);
34         disp("          ", 0, 2);
35         disp("          ", 0, 3);
36         return "Time LE";
37     }
38     if(tdiff<100)
39         return testCS(x, y);
40     else
41     {
42         espCom("plot cnt="+String(cnt)+"\r\n",10);
43         for(register int i=0;i<200;++i)
44             espCom(String(i)+","+String(rcdv[i])+",", 5);
45     }
46     return String(0.149*tdiff/680.0)+"uF ";
47 }

```

5.4 三极管测量实现

首先扫描二极管，在两两端口加压寻找可以导通的端口，可以直接确定是 PNP 还是 NPN 型：如果是某一端口向另外两端导通，则为 NPN；如果是两端向同一端口导通，则为 PNP。其中，共同的端口为基极。在另外两端写入不同电平进行测量，根据原理公式，测得放大倍数较高的值为正偏放大倍数，较小为反偏放大倍数。

以下代码实现了测量操作 NPN 三极管的操作。

```

1 void testNPN(int b)
2 {
3     setLow();
4     disp(String("Mode: NPN"), 0, 1);
5     pinMode(TP[1][ADC], INPUT);
6     pinMode(TP[2][ADC], INPUT);
7     pinMode(TP[3][ADC], INPUT);
8     // first we suppose b+1 is e and b+2 is c
9     int c=1+b%3, e=1+(b+1)%3;
10    pinMode(TP[b][R1], OUTPUT);
11    pinMode(TP[c][Rs], OUTPUT);
12    pinMode(TP[e][Rs], OUTPUT);
13    digitalWrite(TP[b][R1], HIGH);
14    digitalWrite(TP[c][Rs], HIGH);
15    digitalWrite(TP[e][Rs], LOW);
16    int sw;

```

```

17 if(not isValid((analogRead(TP[b][ADC])-analogRead(TP[e][ADC]))/1023.0*5.0)
    ) // block
18 {
19     sw=c, c=e, e=sw;
20     pinMode(TP[c][R1], INPUT);
21     pinMode(TP[e][R1], INPUT);
22     pinMode(TP[c][Rs], OUTPUT);
23     pinMode(TP[e][Rs], OUTPUT);
24     digitalWrite(TP[c][Rs], HIGH);
25     digitalWrite(TP[e][Rs], LOW);
26     if(not isValid((analogRead(TP[b][ADC])-analogRead(TP[e][ADC]))
        /1023.0*5.0))
27     {
28         disp("Invalid", 0, 2);
29         delay(1000);
30         return;
31     }
32 }
33 disp("TP:e"+String(e)+"b"+String(b)+"c"+String(c), 0, 0);
34 delay(50);
35 double vb=analogRead(TP[b][ADC])/1023.0*5.0, vc=analogRead(TP[c][ADC])
    /1023.0*5.0, ve=analogRead(TP[e][ADC])/1023.0*5.0, vcc=5.0;
36 // disp("TP:e"+String(analogRead(TP[e][ADC]))+"b"+String(analogRead(TP[b][
    ADC]))+"c"+String(analogRead(TP[c][ADC])), 0, 3);
37 disp("beta="+String((vcc-vc)/680/((vcc-vb)/470000)), 0, 2);
38 delay(500);
39 }

```

5.5 控制原理

控制函数重复代码多，仅在附录里给出。

控制逻辑的基本要求是：涵盖所有可能的情况，并尽可能避免识别错误导致无法测量的情况。

接下来阐述控制的具体逻辑：

总体流程是判断三极管、判断小电阻、判断短路、判断电容（含断路）、判断大电阻、判断错误。

1. 第一步进行三极管的判断，因为三极管测量时由于电压进行两两端口电压扫描。由于使用小电阻加电平，扫描时间耗时可以控制在 35ms 以内。扫描到上文所述的两个 0.7V 的导通电压，即进入三极管测量函数。如果不优先判断三极管，容易识别成分压为导通电压的电阻或识别为电容。测试三极管的主要时间消耗就是扫描，所以三极管可以秒测，显示停留一段时间方便读数。
2. 第二步测量小电阻。刚才已经测试了两两电压。如果分压在合理的范围，就直接计算输出小电阻值。在测量小电阻时，由于小电阻电压扫描很快，刷新率和灵敏度很高。几乎做到即插即测，拔掉瞬间回复空载。

3. 第三步测量电容。测量电容是扫描前面两两电压为 0 的点进行充放电测试首先进入大电容测试函数，如果瞬间充放电完成，说明电容过小换用大电阻进行小电容测试，如果依然过小，则其实是空载，输出空载。空载判断就是识别为一个十分小的电容。如果所有断路的测试点都是空载，那么就是空载情况，如果测到了电容值，则输出。为了避免插拔导致的电容充电开始，元件却重新插入了电阻中，导致无法充满，设置了最长充电时长。如果超时，输出 Time LE。由于三极管已经排除，不会发生误识别。
测试电容时，还需要讲采样点用数组保存以发送给服务器。实际测试时，发现使用局部变量，数组允许大小过小，而且发现电容的测试结果也受到数组大小明显影响。推测原因是寻址时间加长，导致判断充电完成的周期加长，最后一次测试导致的溢出时间增加，电容测试结果偏大。
4. 第四步，如果有短路点，则输出。短路的测试位置不是十分重要。
5. 第五步测试大电阻。这是因为大电阻测试需要设置更高的延时。测试发现，如果不进行延时，大电阻分压不能达到理论值，而延时后正常。讲大电阻测量放到最后，是考虑了系统的使用体验，否则测量其他元件时，需要等待大电阻完成扫描测试，耗时长。
6. 如果没有进入任何测试（实际上没有发生这样的事），输出错误并重启。

5.6 附加功能

- 电池

我们使用拓展板进行供电，我在上面焊接了电池和锂电池，可以实现可充电、可开关的真无线万用表。

- 显示

我们使用了国外大佬实现的 NOKIA5110_TEXT 库。该库的功能有在指定位置显示字符和字符串。理论上该库支持图像，但经过长时间实验发现该库对点阵图进行了重编码，故启动动画没有使用图像。使用不断变化位置的字符串显示了开机欢迎动画。开机后，显示器上方根据模式显示当前模式、测试结果、提示和警告、测试点信息。测试点信息是三极管的 ebc 极位置或者电容电阻的两端。提示会显示电容测试中，不要拔掉等信息。测试电容时，还会输出充电时间。三极管和电容测试结果有读数延时，小电阻测量时显示器会以高刷新率刷新，大电阻时刷新率降低。

如果连接了服务器，屏幕下方会显示当前时间，并且会和测试结果同步刷新。

- 无线模块

如图 18 连接了无线模块 ESP8266 模块实现 WIFI 连接。使用如下代码实现网络连接。其中，esoCom 函数在附录中，实现的功能是单片机将信息写给 8266，并收取一定时间内的返回信息。

```
1 // wifi connection
2 Serial.begin(baud);
3 esp.begin(baud);
4 delay(5000);
5 code below will be remembered inside esp8266
6 espCom(F("AT+SAVETRANSLINK=1,\"192.168.1.9\",2333,\"TCP\\\"r\\n\""), 2000)
;
```

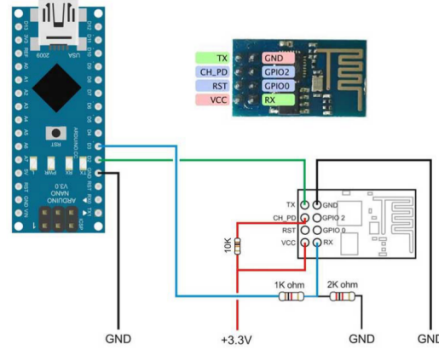


图 18: WIFI 模块连线

```

7  espCom(F("AT+CWMODE=1\r\n"), 2000);
8  espCom(F("AT+RST\r\n"), 2000);
9  espCom(F("AT+CWJAP=\"CMCC-SJTU\", \"Ww331811\"\r\n"), 10000);
10 espCom(F("AT+CIPSTART=\"TCP\", \"192.168.1.9\", 2333\r\n"), 10000);
11 espCom(F("AT+CIPMODE=1\r\n"), 2000);
12 espCom(F("AT+CIPSEND\r\n"), 2000);
13 rt = espCom("Client Started.\r\n", 500);

```

在 ESP8266 中刷入标准 AT 固件，如上的 AT 指令只需执行一次，设置了 8266 储存连接信息，并且上电开启透传模式，即仅作为转发器。每次识别，单片机将缓存发送给服务器，服务器发回当前时间字符串。

服务器使用 python 实现，使用 WIFI 连接，可以实现类似共享单车（共享单车一般使用蓝牙）的远程连接。使用 TCP 协议，可以防止信息丢失。具有多路连接，将万用表与手机进行连接，连接多个万用表等功能（即实现了多线程连接）。服务器收取缓存后，将 log 写在服务器本地。如果发现万用表测试了一次电容，就尝试绘图。但是发现，由于记录需要时间，实际测试的第一个记录点电压已经到了较高的位置，绘图过于离散，于是提交的代码中取消了该功能。

结合服务器和本期储存，以及拓展板上的按钮，还可以实现历史记录调取的功能，菜单指定模式等，思路简单，没有给出实现。

5.7 精度和 BUG 分析

测试结果于附件表格给出。电阻测试精度较高。在测试了足够多的电阻后，使用理想计算公式图线存在系统偏差，在原式中添加参数整体移动了图线使之准确。可惜的是，由于是电阻数量不够多，无法细调。为了避免凑数嫌疑，保留了一定误差。电容存在类似情况。电容误差还与代码量有关联，发现增删无关代码影响电容测试结果，匪夷所思。推测是由于采用了自己购买的芯片，时钟不稳定。实际测试时，发现时钟差到 288ns 时才会发生跳变，可能也与之有关。这也影响电容的准确程度。三极管按照理想电路进行了测试，由于没有标准测试仪，我们本着诚实的态度，输出了于参数误差极大的放大倍数，没有进行修改。但是，我们测试了温度影响下的放大倍数变化，证明我们测试的结果确实与放大倍数正相关。

在测试过程中，随意插拔，可能导致显示屏幕花屏。但是从服务器运行状态上看，测试在正常进行，推测为国外大佬显示器代码 bug。此外，也有小概率发生单片机死机重启，但比较难以触发，故不纠其原因。

6 团队分工

王凯灵：前期实现了超声波电路。设计实现了万用表电路和代码以及拓展功能，报告撰写相应部分。

代铭波：元件测量仪部分理论和设计思路推导，模拟仿真，报告撰写相关部分。

张杭磊：原理部分的撰写，报告内容与图片的排版。

7 附录

7.1 嵌入式代码

```
1 // define the LCD screen
2 #include <NOKIA5110_TEXT.h>
3 NOKIA5110_TEXT display(15, 16, 17, 7, 6);
4 #define inverse false
5 #define UseDefaultFont false
6 #define contrast 0xBF // default is 0xBF set in LCDinit, Try 0xB1(good @ 3.3V) or 0xBF
    if your display is too dark
7 #define bias 0x13 // LCD bias mode 1:48: Try 0x13 or 0x14
8 #define cd display.LCDClear();
9 // define the 8266 module
10 #include <SoftwareSerial.h>
11 SoftwareSerial esp(2, 3);
12 uint32_t baud = 115200;
13 String rt = "";
14 // control bits for testpoints
15 int TP[4][3]{0, 0, 0, 8, 9, 14, 12, 13, 19, 10, 11, 18};
16 #define Rs 0 // small R = 680 Ohm
17 #define Rl 1 // large R = 470 kOhm
18 #define ADC 2 // ADC input(PCx)
19
20 #define msg String(rcd[0][0][1])+","+String(rcd[0][0][2])+","+String(rcd[0][1][0])+","+
    String(rcd[0][1][2])+","+String(rcd[0][2][0])+","+String(rcd[0][2][1])+","+String(
    rcd[1][0][1])+","+String(rcd[1][0][2])+","+String(rcd[1][1][0])+","+String(rcd
    [1][1][2])+","+String(rcd[1][2][0])+","+String(rcd[1][2][1])+"\r\n"
21
22 void setup()
23 {
24     // screen init and boot effect
25     display.LCDInit(inverse, contrast, bias); //initialize
26     cd //clear whole screen
27
28     // wifi connection
```

```

29 Serial.begin(baud);
30 esp.begin(baud);
31 delay(5000);
32 // code below will be remembered inside esp8266
33 // espCom(F("AT+SAVETRANSLINK=1,\"192.168.1.9\",2333,\"TCP\"\\r\\n\"), 2000);
34 // espCom(F("AT+CWMODE=1\\r\\n\"), 2000);
35 // espCom(F("AT+RST\\r\\n\"), 2000);
36 // espCom(F("AT+CWJAP=\"CMCC-SJTU\", \"Ww331811\"\\r\\n\"), 10000);
37 // espCom(F("AT+CIPSTART=\"TCP\", \"192.168.1.9\",2333\\r\\n\"), 10000);
38 // espCom(F("AT+CIPMODE=1\\r\\n\"), 2000);
39 // espCom(F("AT+CIPSEND\\r\\n\"), 2000);
40 onboot_notice();
41 rt = espCom("Client Started.\\r\\n", 500);
42 }
43
44 void loop()
45 {
46   if(rt[0]=='t')
47     rt[0] = ' ', disp(rt+" ", 0, 4);
48   rt = espCom(autoSelect(), 333);
49
50 }
51
52 // simple display controller
53 void disp(String s, int X, int Y)
54 {
55   display.LCDgotoXY(X, Y); // (go to (X , Y) (0-84 columns, 0-5 blocks)
56   display.LCDString(&s[0]); //print
57 }
58
59 // onboot animation
60 void onboot_notice()
61 {
62   for(register int i=0; i<6; ++i)
63   {
64     disp("114", 15, i);
65     disp("514", 55, 5-i);
66     delay(300);
67   }
68   cd
69   disp("Welcome", 20, 2);
70   disp("to use", 25, 3);
71   disp("LOPING151", 15, 5);
72   delay(3000);
73   cd
74   display.LCDgotoXY(0, 0);
75   display.LCDString("Connecting");
76   display.LCDgotoXY(0, 1);
77   display.LCDString("to server");

```

```

78  display.LCDgotoXY(0, 2);
79  delay(2000);
80  cd
81  }
82
83  // send command or data to the server
84  String espCom(String command, const int timeout)
85  {
86      String inData = "";
87      esp.print(command);
88      long int time = millis();
89      while ( (time + timeout) > millis())
90      {
91          while (esp.available())
92          {
93              char c = esp.read();
94              inData += c;
95          }
96      }
97      return inData;
98  }
99
100 // set all pin Low before each test
101 void setLow()
102 {
103     for(register int i=1;i<=3;++i)
104         for(register int j=0;j<=2;++j)
105             pinMode(TP[i][j], INPUT);
106 }
107
108 // test the case for small resistance
109 double testRS(int x, int y, int cmd=1)
110 {
111     setLow();
112     pinMode(TP[x][ADC], INPUT);
113     pinMode(TP[y][ADC], INPUT);
114     pinMode(TP[x][R1], INPUT);
115     pinMode(TP[y][R1], INPUT);
116     pinMode(TP[x][Rs], OUTPUT);
117     pinMode(TP[y][Rs], OUTPUT);
118     digitalWrite(TP[x][Rs], HIGH);
119     digitalWrite(TP[y][Rs], LOW);
120     delay(5);
121     int vintH=analogRead(TP[x][ADC]);
122     int vintL=analogRead(TP[y][ADC]);
123     if(abs(vintH+vintL-1023)>5 and cmd)
124         return 114;
125     double vdiff = (vintH-vintL)*5.0/1023;
126     return vdiff;

```

```

127 }
128
129 // test the case for large resistance
130 double testRL(int x, int y)
131 {
132     setLow();
133     pinMode(TP[x][ADC], INPUT);
134     pinMode(TP[y][ADC], INPUT);
135     pinMode(TP[x][Rs], INPUT);
136     pinMode(TP[y][Rs], INPUT);
137     pinMode(TP[x][R1], OUTPUT);
138     pinMode(TP[y][R1], OUTPUT);
139     digitalWrite(TP[x][R1], HIGH);
140     digitalWrite(TP[y][R1], LOW);
141     delay(50);
142     int vinth=analogRead(TP[x][ADC]);
143     int vintl=analogRead(TP[y][ADC]);
144     if(vinth<850)
145         delay(200);
146     else
147         return 514;
148     vinth=analogRead(TP[x][ADC]);
149     vintl=analogRead(TP[y][ADC]);
150     if(abs(vinth+vinth-1023)>15)
151         return 514;
152     double vdiff = (vinth-vintl)*5.0/1023;
153     return vdiff;
154 }
155
156 // large capacity
157 String testCL(int x, int y)
158 {
159     setLow();
160     disp(String("Mode: LC"), 0, 1); // Small C
161     disp(String("Testing"), 0, 2);
162     disp(String("Don't shift!"), 0, 3); // Warning
163     pinMode(TP[x][ADC], INPUT);
164     pinMode(TP[y][ADC], INPUT);
165     pinMode(TP[x][R1], INPUT); // use small R, the charge time is RC. If C is too small,
        charge time is small. This case goto TestCS to use larger R
166     pinMode(TP[y][R1], INPUT);
167     pinMode(TP[x][Rs], OUTPUT);
168     pinMode(TP[y][Rs], OUTPUT);
169     int rcdv[200];
170     String tmpS;
171     int cnt=0;
172     int vinth=analogRead(TP[x][ADC]);
173     int vintl=analogRead(TP[y][ADC]);
174     while(vinth>5 or vintl>5)

```

```

175     vintl=analogRead(TP[x][ADC]), vintl=analogRead(TP[y][ADC]); // wait until q=0
176     delay(10);
177     long int Cstart=micros();
178     digitalWrite(TP[x][Rs], HIGH);
179     while(analogRead(TP[x][ADC])<1018 and micros()-Cstart<1000000)
180         rcdv[max(cnt++/3,200)]=analogRead(TP[x][ADC]);
181     long int tdiff=micros()-Cstart-116; // 116 is the base
182     tdiff=abs(tdiff);
183     disp("          ", 0, 3);
184     disp("Used "+String(tdiff)+"us",0,3);
185     if(tdiff>900000)
186     {
187         disp("          ", 0, 0);
188         disp("          ", 0, 1);
189         disp("          ", 0, 2);
190         disp("          ", 0, 3);
191         return "Time LE";
192     }
193     if(tdiff<100)
194         return testCS(x, y);
195     else
196     {
197         espCom("plot cnt="+String(cnt)+"\r\n",10);
198         for(register int i=0;i<200;++i)
199             espCom(String(i)+","+String(rcdv[i])+",", 5);
200     }
201     return String(0.149*tdiff/680.0)+"uF ";
202 }
203
204 // small capacity
205 String testCS(int x, int y)
206 {
207     setLow();
208     disp(String("Mode: SC"), 0, 1); // Large C
209     pinMode(TP[x][ADC], INPUT);
210     pinMode(TP[y][ADC], INPUT);
211     pinMode(TP[x][Rs], INPUT);
212     pinMode(TP[y][Rs], INPUT);
213     pinMode(TP[x][R1], OUTPUT);
214     pinMode(TP[y][R1], OUTPUT);
215     int vintl=analogRead(TP[x][ADC]);
216     int vintl=analogRead(TP[y][ADC]);
217     while(vinth>2 or vintl>2)
218         vintl=analogRead(TP[x][ADC]), vintl=analogRead(TP[y][ADC]); // wait until q=0
219     long int Cstart=micros();
220     digitalWrite(TP[x][R1], HIGH);
221     while(analogRead(TP[x][ADC])<1020 and micros()-Cstart<344);
222     long int tdiff=micros()-Cstart-344;
223     tdiff=abs(tdiff);

```

```

224     disp("          ", 0, 3);
225     disp("Used "+String(tdiff)+"us",0,3);
226     if(tdiff>900000)
227     {
228         disp("          ", 0, 0);
229         disp("          ", 0, 1);
230         disp("          ", 0, 2);
231         disp("          ", 0, 3);
232         return "Time LE";
233     }
234     if(tdiff<220)
235     {
236         disp("          ", 0, 0);
237         disp("          ", 0, 1);
238         disp("          ", 0, 2);
239         disp("          ", 0, 3);
240         disp("Empty load", 0, 0);
241         delay(500);
242         return "";
243     }
244     return String(0.1*tdiff/470.0*1000)+"pF ";
245 }
246
247 // judge if the voltage is valid for triod
248 bool isValid(double x)
249 {
250     return x>0.4 and x<1.2;
251 }
252
253 void testNPN(int b)
254 {
255     setLow();
256     disp(String("Mode: NPN"), 0, 1);
257     pinMode(TP[1][ADC], INPUT);
258     pinMode(TP[2][ADC], INPUT);
259     pinMode(TP[3][ADC], INPUT);
260     // first we suppose b+1 is e and b+2 is c
261     int c=1+b%3, e=1+(b+1)%3;
262     pinMode(TP[b][R1], OUTPUT);
263     pinMode(TP[c][Rs], OUTPUT);
264     pinMode(TP[e][Rs], OUTPUT);
265     digitalWrite(TP[b][R1], HIGH);
266     digitalWrite(TP[c][Rs], HIGH);
267     digitalWrite(TP[e][Rs], LOW);
268     int sw;
269     if(not isValid((analogRead(TP[b][ADC])-analogRead(TP[e][ADC]))/1023.0*5.0)) // block
270     {
271         sw=c, c=e, e=sw;
272         pinMode(TP[c][R1], INPUT);

```

```

273     pinMode(TP[e][R1], INPUT);
274     pinMode(TP[c][Rs], OUTPUT);
275     pinMode(TP[e][Rs], OUTPUT);
276     digitalWrite(TP[c][Rs], HIGH);
277     digitalWrite(TP[e][Rs], LOW);
278     if(not isValid((analogRead(TP[b][ADC])-analogRead(TP[e][ADC]))/1023.0*5.0))
279     {
280         disp("Invalid", 0, 2);
281         delay(1000);
282         return;
283     }
284 }
285 disp("TP:e"+String(e)+"b"+String(b)+"c"+String(c), 0, 0);
286 delay(50);
287 double vb=analogRead(TP[b][ADC])/1023.0*5.0, vc=analogRead(TP[c][ADC])/1023.0*5.0, ve=
    analogRead(TP[e][ADC])/1023.0*5.0, vcc=5.0;
288 // disp("TP:e"+String(analogRead(TP[e][ADC]))+"b"+String(analogRead(TP[b][ADC]))+"c"+
    String(analogRead(TP[c][ADC])), 0, 3);
289 disp("beta="+String((vcc-vc)/680/((vcc-vb)/470000)), 0, 2);
290 delay(500);
291 }
292
293 void testPNP(int b)
294 {
295     setLow();
296     disp(String("Mode: PNP"), 0, 1);
297     pinMode(TP[1][ADC], INPUT);
298     pinMode(TP[2][ADC], INPUT);
299     pinMode(TP[3][ADC], INPUT);
300     // first we suppose b+1 is e and b+2 is c
301     int c=1+b%3, e=1+(b+1)%3;
302     pinMode(TP[b][R1], OUTPUT);
303     pinMode(TP[c][Rs], OUTPUT);
304     pinMode(TP[e][Rs], OUTPUT);
305     digitalWrite(TP[b][R1], LOW);
306     digitalWrite(TP[c][Rs], LOW);
307     digitalWrite(TP[e][Rs], HIGH);
308     int sw;
309     if(not isValid(-(analogRead(TP[b][ADC])-analogRead(TP[e][ADC]))/1023.0*5.0)) // block
310     {
311         sw=c, c=e, e=sw;
312         pinMode(TP[c][R1], INPUT);
313         pinMode(TP[e][R1], INPUT);
314         pinMode(TP[c][Rs], OUTPUT);
315         pinMode(TP[e][Rs], OUTPUT);
316         digitalWrite(TP[c][Rs], LOW);
317         digitalWrite(TP[e][Rs], HIGH);
318         if(not isValid(-(analogRead(TP[b][ADC])-analogRead(TP[e][ADC]))/1023.0*5.0))
319         {

```

```

320     disp("Invalid", 0, 2);
321     delay(1000);
322     return;
323 }
324 }
325 disp("TP:e"+String(e)+"b"+String(b)+"c"+String(c), 0, 0);
326 delay(50);
327 double vb=analogRead(TP[b][ADC])/1023.0*5.0, vc=analogRead(TP[c][ADC])/1023.0*5.0, ve=
    analogRead(TP[e][ADC])/1023.0*5.0, vcc=5.0;
328 // disp("TP:e"+String(analogRead(TP[e][ADC]))+"b"+String(analogRead(TP[b][ADC]))+"c"+
    String(analogRead(TP[c][ADC])), 0, 3);
329 disp("beta="+String(vc/680/(vb/470000)), 0, 2);
330 delay(500);
331 }
332
333 // do tests and auto select the mode
334 String autoSelect()
335 {
336     disp("      ", 0, 0);
337     disp("      ", 0, 1);
338     disp("      ", 0, 2);
339     disp("      ", 0, 3);
340
341     double rcdt[3][3]{};
342     for(register int i=0;i<9;++i)
343         if(i%3!=i/3)
344             rcdt[i/3][i%3]=testRS(i/3+1, i%3+1, 0);
345     // Cases for Triode
346     if(isValid(rcdt[1][0]) and isValid(rcdt[2][0]))
347     {
348         testPNP(1);
349         return "-1\r\n";
350     }
351     if(isValid(rcdt[0][1]) and isValid(rcdt[2][1]))
352     {
353         testPNP(2);
354         return "-1\r\n";
355     }
356     if(isValid(rcdt[0][2]) and isValid(rcdt[1][2]))
357     {
358         testPNP(3);
359         return "-1\r\n";
360     }
361     if(isValid(rcdt[0][1]) and isValid(rcdt[0][2]))
362     {
363         testNPN(1);
364         return "-1\r\n";
365     }
366     if(isValid(rcdt[1][0]) and isValid(rcdt[1][2]))

```

```

367 {
368     testNPN(2);
369     return "-1\r\n";
370 }
371 if(isValid(rcdt[2][0]) and isValid(rcdt[2][1]))
372 {
373     testNPN(3);
374     return "-1\r\n";
375 }
376
377 double rcd[2][3][3]{}; // rcd[0] records the results for testRS and rcd[1] for testRL.
378
379 // Cases for R
380 for(register int i=0;i<9;++i)
381     if(i%3!=i/3)
382     {
383         rcd[0][i/3][i%3]=testRS(i/3+1, i%3+1);
384         if(rcd[0][i/3][i%3]>=0.2 and rcd[0][i/3][i%3]<4.5)
385         {
386             disp(String("TP on ") + String(i/3+1) + String("-") + String(i%3+1), 0, 0);
387             disp(String("Mode: SR"), 0, 1); // Small R
388             disp(String(1.035*1360*rcd[0][i/3][i%3]/(5-rcd[0][i/3][i%3])) + String(" Ohm"), 0,
389                 2);
389             return msg;
390         }
391     }
392
393 // Cases for short
394 if(rcd[0][0][1]<0.1 or rcd[0][0][2]<0.1 or rcd[0][1][2]<0.1 or rcd[0][1][0]<0.1 or rcd
395     [0][2][0]<0.1 or rcd[0][2][1]<0.1)
396 {
397     disp(String("Short"),0, 0), delay(2000);
398     return msg;
399 }
400
401 for(register int i=0;i<9;++i)
402     if(i%3!=i/3)
403     {
404         rcd[1][i/3][i%3]=testRL(i/3+1, i%3+1);
405         if(rcd[1][i/3][i%3]>=0.1 and rcd[1][i/3][i%3]<=4.8)
406         {
407             disp(String("TP on ") + String(i/3+1) + String("-") + String(i%3+1), 0, 0);
408             disp(String("Mode: LR"), 0, 1); // Large R
409             disp(String(-8+1.233*940*rcd[1][i/3][i%3]/(5-rcd[1][i/3][i%3])) + String(" kOhm"),
410                 0, 2);
411             return msg;
412         }
413     }
414 }
415

```

```

413 // Cases for C
414 if((rcd[0][0][1]>4.5 or rcd[0][1][0]>4.5) and (max(rcd[0][0][1], rcd[0][1][0])>=max(
    rcd[0][0][2], rcd[0][2][0])) and (max(rcd[0][0][1], rcd[0][1][0])>=max(rcd
    [0][1][2], rcd[0][2][1])))
415 {
416     disp(String("TP on ")+String(1)+String("-")+String(2), 0, 0);
417     String resultC = testCL(1, 2);
418     if(resultC!="")
419     {
420         disp(resultC, 0, 2);
421         delay(2000);
422         cd
423     }
424     return msg;
425 }
426 if((rcd[0][0][2]>4.5 or rcd[0][2][0]>4.5) and (max(rcd[0][0][2], rcd[0][2][0])>=max(
    rcd[0][0][1], rcd[0][1][0])) and (max(rcd[0][0][2], rcd[0][2][0])>=max(rcd
    [0][1][2], rcd[0][2][1])))
427 {
428     disp(String("TP on ")+String(1)+String("-")+String(3), 0, 0);
429     String resultC = testCL(1, 3);
430     if(resultC!="")
431     {
432         disp(resultC, 0, 2);
433         delay(2000);
434         cd
435     }
436     return msg;
437 }
438 if((rcd[0][1][2]>4.5 or rcd[0][2][1]>4.5) and (max(rcd[0][1][2], rcd[0][2][1])>=max(
    rcd[0][0][2], rcd[0][2][0])) and (max(rcd[0][1][2], rcd[0][2][1])>=max(rcd
    [0][0][1], rcd[0][1][0])))
439 {
440     disp(String("TP on ")+String(2)+String("-")+String(3), 0, 0);
441     String resultC = testCL(2, 3);
442     if(resultC!="")
443     {
444         disp(resultC, 0, 2);
445         delay(2000);
446         cd
447     }
448     return msg;
449 }
450
451
452 // shouldn't reach here
453 cd
454 disp("Error", 0, 0);
455 delay(5000);

```

```
456     return msg;
457 }
```

8 服务器端代码

```
1 import socket
2 import threading
3 import matplotlib.pyplot as plt
4 from time import sleep, time, strftime, localtime
5 from sys import exit
6 import numpy as np
7 from csv import reader
8
9 plt.rcParams['font.sans-serif'] = ['SimHei'] # 表格中文
10 plt.rcParams['axes.unicode_minus'] = False
11
12 # 记录各个部分连接的的端口
13 port_esp = 2333
14 port_mobile = 2233
15
16 # 建项目文件夹同路径下的文件夹，用于保存数据并记录时间
17 file_count = 0 # 当前文件数目
18 str_time = strftime('%Y-%m-%d, %H.%M.%S', localtime()) # 当前时间
19 dir_name = '../user_data/' + str_time
20 filename = ''
21
22
23 # mkdir(dir_name) # 按照时间创建文件夹
24
25
26 # 画图的函数
27 def graph():
28     sampling_rate = 125
29     plt_len = 6
30     global package_count
31     # 先建立窗口
32     if package_count == 0:
33         plt.ion()
34         plt.figure(figsize=(12, 5))
35     else:
36         if package_count % 1 == 0: # 每几包画一个
37             plot_data = list()
38             with open(filename, 'r') as file:
39                 csv_reader = reader(file)
40                 for row in csv_reader:
41                     plot_data.append(float(row[0].strip()))
```

```

42         if len(plot_data) <= sampling_rate * plt_len:
43             plt.plot(plot_data)
44         else:
45             plt.plot(plot_data[len(plot_data) - sampling_rate *
46                             plt_len:len(plot_data)])
47         plt.title('波形')
48         plt.xlabel('采样点')
49         plt.ylabel('电压值')
50         plt.axis([0, sampling_rate * plt_len, -2, 4])
51         plt.pause(0.1)
52         plt.clf()
53
54 # 文件保存函数，用于数据分割成多文件保存
55 def file_save(start_time):
56     global file_count, filename
57     total_package = 1000 # 程序停止时记录的包数
58     package_length = 6 # 一包数据时长
59     if file_count > total_package:
60         exit(0) # 程序出口
61     if time() - start_time >= package_length:
62         if file_count and file_count != 2:
63             print("Unlocking.....")
64             sleep(0.2)
65         else:
66             if file_count and file_count != 2:
67                 print('\nIdentification unsuccessful.')
68             file_count += 1
69             filename = dir_name + '/file' + str(file_count) + '.csv'
70             new_f = open(filename, 'w', newline='')
71             print('\nSaving data in file' + str(file_count), end='')
72             return new_f, time()
73     else:
74         if package_length > 10:
75             graph()
76         f = open(filename, 'a', newline='')
77         return f, start_time
78
79
80 def handle_mobile(c_mobile, add_mobile):
81     print("Client mobile:", add_mobile, "connected")
82     while True:
83         msg = ""
84         c_mobile.send(msg.encode())
85         sleep(0.2)
86         msg = ""
87         c_mobile.send(msg.encode())
88
89

```

```

90 def handle_esp(c_esp, add_esp):
91     print("Client ESP8266:", add_esp, "connected")
92     while True:
93         data = c_esp.recv(2 ** 15).decode('utf-8')
94         print(data, end='')
95         if data[:4] == "plot":
96             s = []
97             for _ in range(800):
98                 s.append(c_esp.recv(2 ** 15).decode('utf-8'))
99             s = np.array(s).flatten()
100             msg = strftime('%Y-%m-%d, %H.%M.%S', localtime())
101             c_esp.send(msg.encode())
102
103
104 # use while for multi-connection
105 print("Server started. Waiting for connection")
106 s_esp = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
107 s_esp.bind(("0.0.0.0", port_esp))
108 s_esp.listen()
109 client_esp, address_esp = s_esp.accept()
110 t1 = threading.Thread(target=handle_esp, args=(client_esp, address_esp))
111 t1.start()
112
113 s_mobile = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
114 s_mobile.bind(("0.0.0.0", port_mobile))
115 s_mobile.listen()
116 client_mo, address_mo = s_mobile.accept()
117 t2 = threading.Thread(target=handle_mobile, args=(client_mo, address_mo))
118 t2.start()

```